

SEMESTER III.

MODULE 5

TREE

Definition :Tree

A connected graph which does not have cycle (or circuit) is called as tree .

Example:



1. Tree is simple connected graph with No cycles.
2. It is always Bipartite graph.
3. Maximally Non-cycle, minimally connected; Adding edge will get cycle, removing edge will disconnect it.
4. In a tree we can always find a path joining any two vertices because it is a connected graph. Such a path is unique, otherwise it will be a cycle.

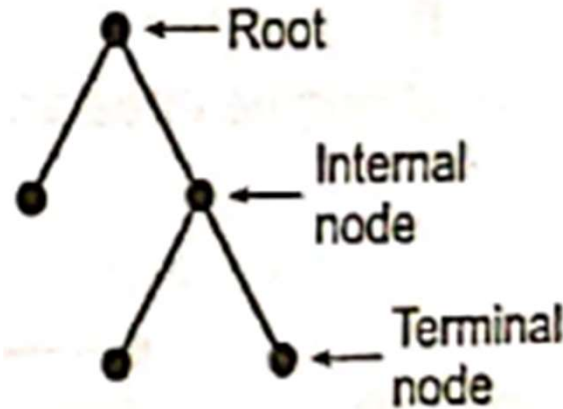
Terminal node/ leaf

A vertex of degree 1 in tree is called as Terminal node or a leaf .

Internal node / Branch Node

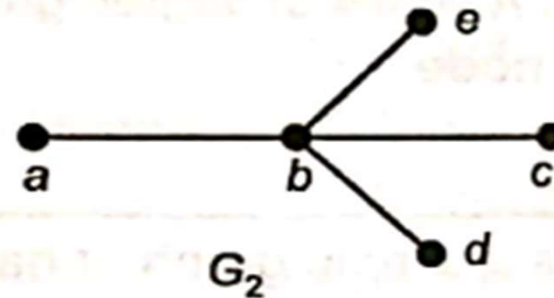
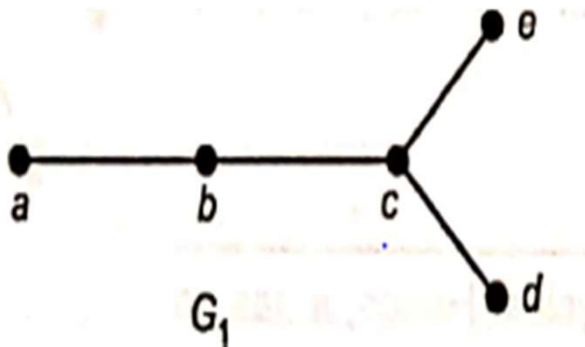
A vertex of degree grater than 1 in tree is called as internal node or branch node .

Example :



Example :

Are following trees isomorphic ?



G_1 and G_2 are not isomorphic trees, Since graph G_2 contains only one vertex of degree 4 and G_1 does not have vertex of degree 4

Properties of Trees

Theorem 1 : Graph is tree if and only if there is one & only one path between every pair of vertices

Theorem 2 : A tree with n vertices has $(n-1)$ edges

Theorem 3 : A connected graph with n vertices having $(n-1)$ edges is tree

Theorem 4 : In any tree with two or more vertices, there are at least 2 pendant vertices.

Example: A tree T has m vertices of degree one, two vertices of degree two, three vertices of degree four and four vertices of degree three then find m .

$$n = m + 2 + 3 + 4 = m + 9 \quad \dots\dots\dots (1)$$

Further, the sum of degrees in T

$$\begin{aligned} &= 1 \times m + 2 \times 2 + 3 \times 4 + 4 \times 3 \\ &= m + 28 \quad \dots\dots\dots (2) \end{aligned}$$

$$\text{i.e.,} \quad \sum d(v_i) = m + 28$$

Let the tree T have k number of edges.

Since T is a tree with n vertices, $k = n - 1$ (See Theorem 3 above).

But the sum of degrees of all vertices of a graph is equal to twice the number of edges.

$$\therefore \quad \sum d(v_i) = 2k = 2(n - 1)$$

Now, using (1),

$$\begin{aligned} \sum d(v_i) &= 2n - 2 = 2(m + 9) - 2 \\ \therefore \quad \sum d(v_i) &= 2m + 16 \quad \dots\dots\dots (3) \end{aligned}$$

From (2) and (3), we get

$$m + 28 = 2m + 16 \quad \therefore m = 12.$$

$\therefore$ There are twelve vertices of degree one.

Spanning Tree

Definition: Let G be a graph. A sub-graph G' of G is called a spanning sub-graph if G and G' have the same set of vertices.

In particular if e is an edge in G , then $G \sim e$ is a spanning sub-graph of G .

Clearly there may be several sub-graphs of G which span G .

However, we are interested in one sub-graph of G called spanning tree.

Definition: Let G be a connected graph. A spanning sub-graph of G which is a tree is called a spanning tree of G .

Spanning Tree

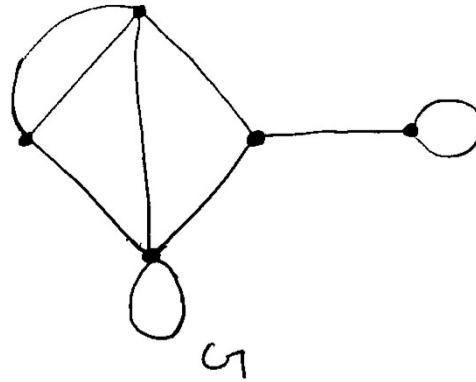
For every connected graph G , a spanning tree exists.

Steps to get a spanning Tree

1. We first remove all the loops of the graph.
2. Then we break a cycle keeping the resulting sub-graph still connected, by removing an edge of the cycle.
3. Continuing the process, till all the cycles are removed we get a spanning tree.

Example

Find the spanning trees of the graph shown below



Solution:

The given graph has $n = 5$ vertices and $e = 9$ edges.

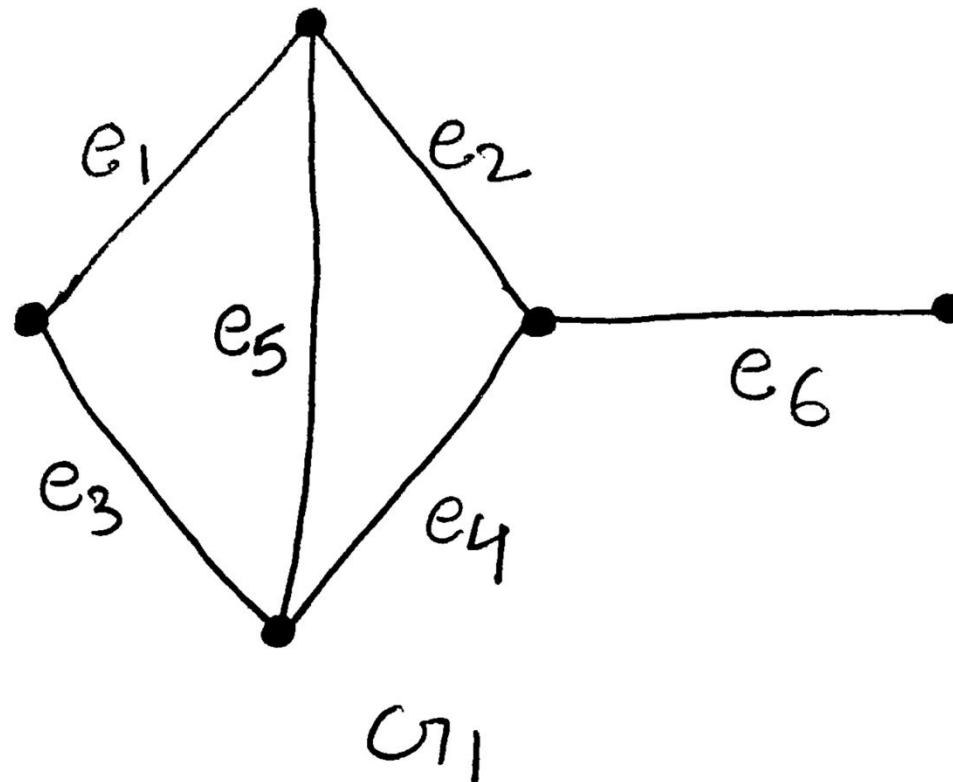
Every spanning tree will have $n = 5$ vertices and $n - 1 = 4$ edges.

Hence we have to remove $9 - 4 = 5$ edges.

We first remove the two loops and multiple edge and get the graph shown below.

Example

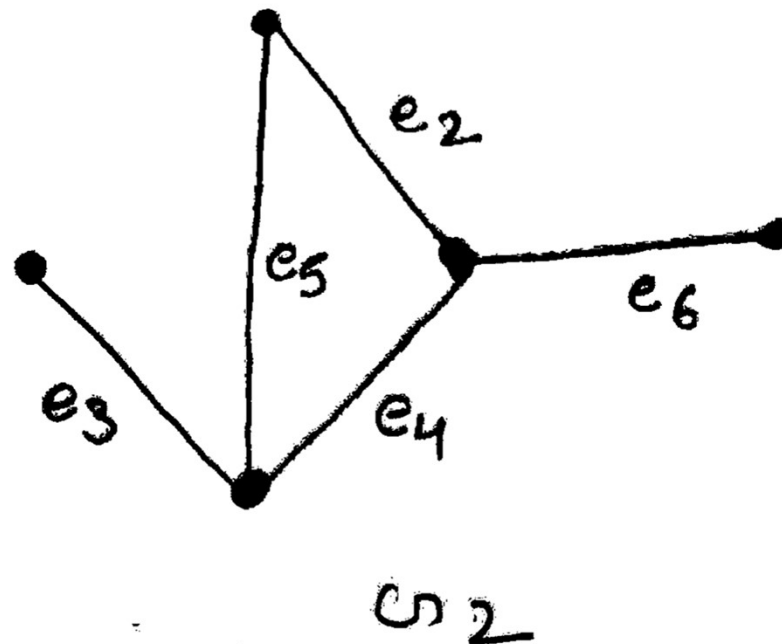
We first remove the two loops and multiple edge and get the graph shown below.



Example

Then we remove the edges one by one to break the cycles.

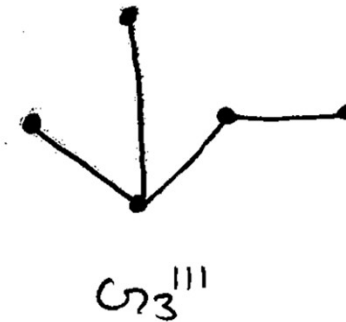
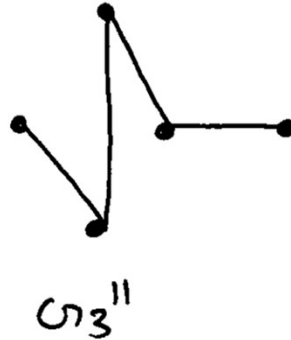
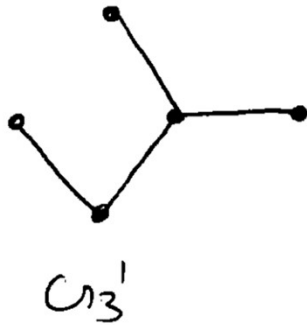
We first remove the edge e_1 to break the cycle on the left and get the following graph.



Example

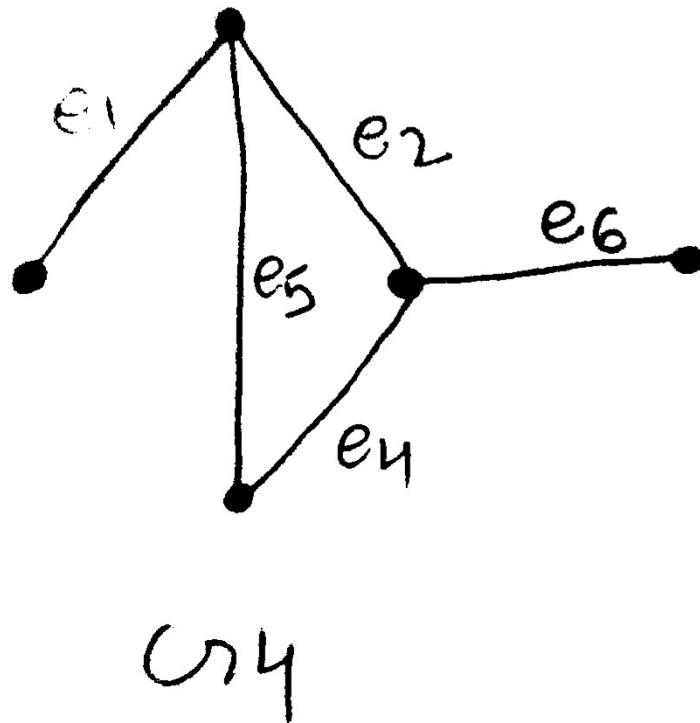
Now we remove the edges e_5 or e_4 or e_2 to break the second cycle .

Thus we get 3 different subgraphs.



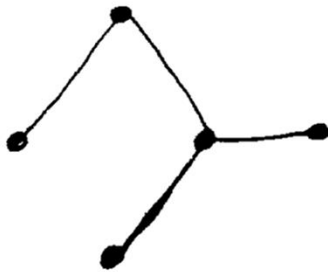
Example

Further, to break the first cycle instead of removing the edge e_1 from the subgraph G_1 , we may remove the edge e_3 and get the graph shown below.

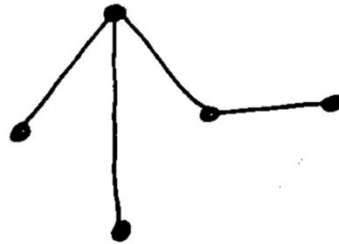


Example

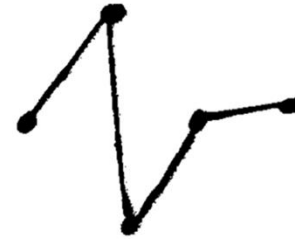
To break the second cycle we may remove as before the edge e_5 or e_4 or e_2 . Thus, we get the following three more trees.



G_4'



G_4''



G_4'''

Example

Note that the spanning trees G_3' and G_4' are isomorphic and so are G_3'' and G_4''' and so are G_3''' and G_4'' .

We can find other spanning trees by removing from the graph G_1 , the edge e_2 or e_4 and then breaking the second cycle.

Minimal Spanning Tree (M.S.T.)

Motivation

Suppose there are n cities to be connected by roads or by railways or by air. Suppose the cost of connecting two cities v and v' is $C(v, v')$.

The problem is to find the way in which the cities should be connected, so that the cost is minimum.

This is known as “**the connector problem**”.

The connector problem can be recast into a problem of graph theory as follows:

Let the cities be denoted by vertices. If two cities v and v' are to be connected then we say that there is an edge e between v and v' . If these edges are $e_1, e_2, e_3, \dots$. Then we have a graph $G = (V, E)$.

Let $C(e)$ denote the cost of constructing the road e .

The connector problem now becomes the problem of finding a spanning tree of the graph G such that the total cost of constructing the roads is minimum.

Minimal Spanning Tree (M.S.T.)

Definition:

Let $G = (V, E)$ be a connected graph. Let $e \in E$ and $C(e)$ be a non negative number associated with e . It is called the **cost** or the **weight** of e . When costs or weights thus are associated with all edges the graph is called the **weighted graph**.

A spanning tree T of graph G such that the total cost $\sum C(e)$ is minimum is called a **minimal spanning tree** (MST) of the graph G .

The sum $\sum C(e)$ for the MST is called the **minimum cost** or the **minimum weight** of the graph G .

There are various algorithms to find MST and the weight of G . We shall study one of them, known as **Kruskal's algorithm**.

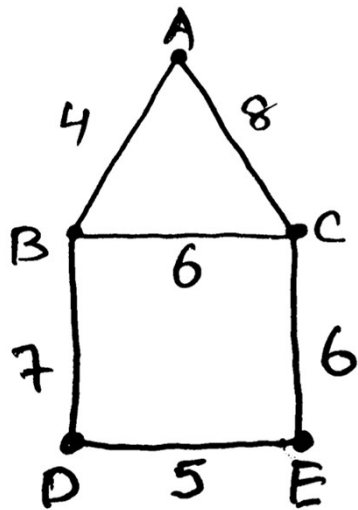
Kruskal's Algorithm

- Suppose we are given a connected weighted graph. First of all name all the vertices . Also list all the edges in the increasing order of their weights (i.e. taking the minimum cost first)
- Now, jot down all the vertices along with their names given in the graph.
- Step 1: Get the first edge by joining the vertices topping the above list.
- Step 2: Get the second edge by joining the vertices second in the list.
- Step 3: consider the next (3rd) edge in the list. If it does not form a cycle with already selected edges then join the vertices to get the edge. If it forms a cycle with the already selected edge, reject it.
- Step 4 : Repeat the above steps till you get a spanning tree i.e. you have joined all the vertices.

This is the required Minimal Spanning Tree.

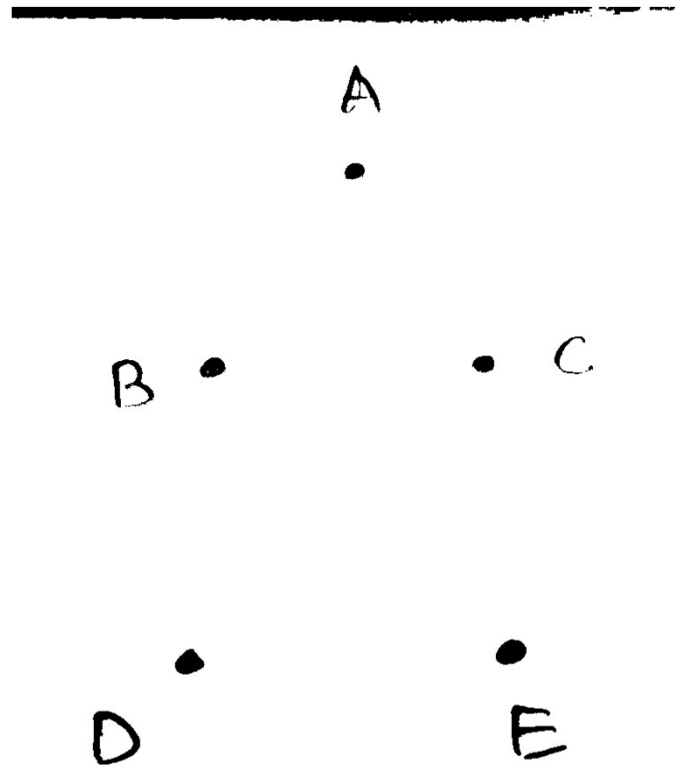
Example-1

Find the spanning tree of the graph shown below with minimum weight.



Example-1

Solution: We first jot down all the vertices as shown in figure below

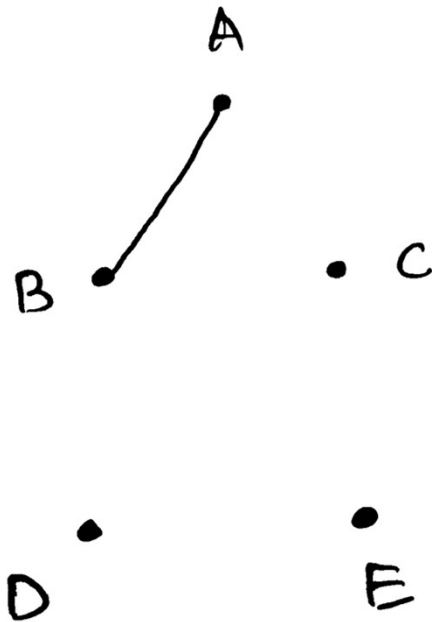


Example-1

We now list the edges in increasing order of their costs (weights)

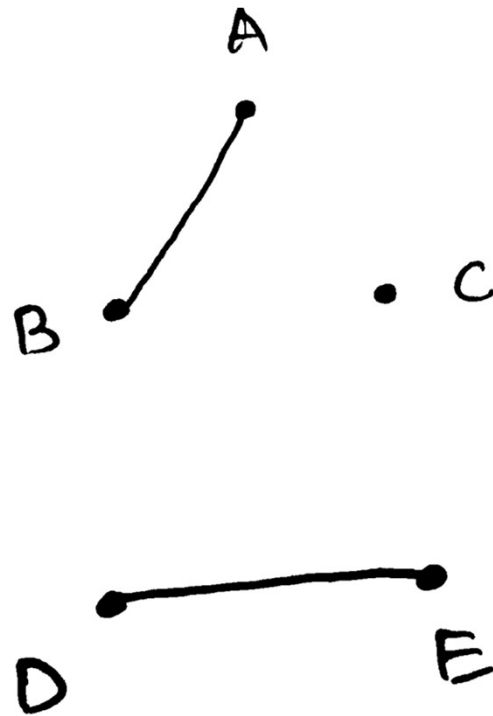
(1) $\{A, B\}$ (2) $\{D, E\}$ (3) $\{E, C\}$ (4) $\{C, B\}$ (5) $\{B, D\}$ (6) $\{A, C\}$

STEP 1: We obtain the first edge by joining the vertices topping the above list . We join the vertices A and B .



Example-1

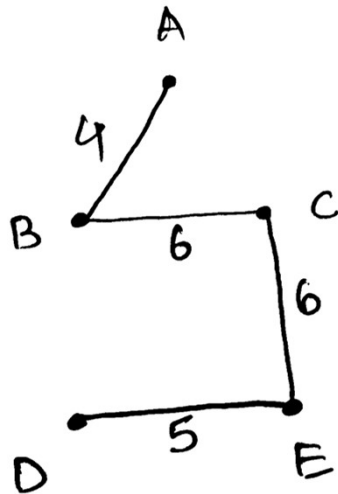
STEP 2: we obtain the second edge by joining the vertices second in the list. We join the vertices D and E



Example-1

STEP 3: we obtain the third edge by joining the vertices third in the order. There are two pairs of vertices having weight 6.

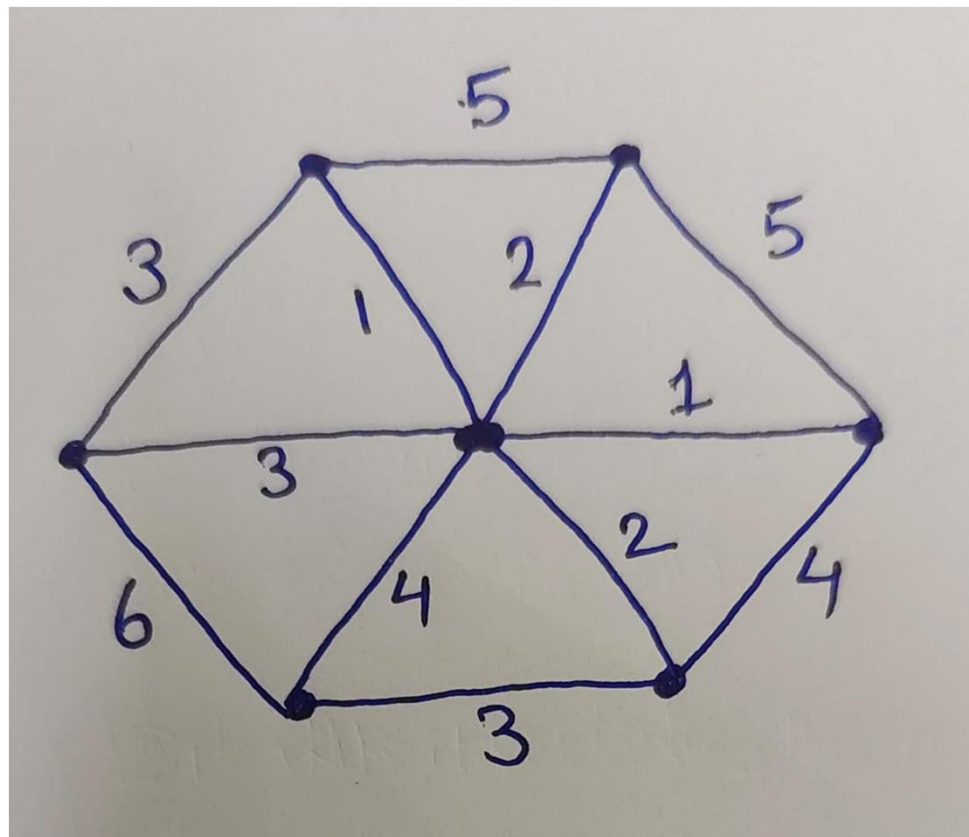
Since they do not form a cycle, we join both these pairs. We thus get the edges EC and CB .



Since all the vertices are now joined, this is the required minimum spanning tree. By adding the costs, we get the minimal cost $4 + 5 + 6 + 6 = 21$

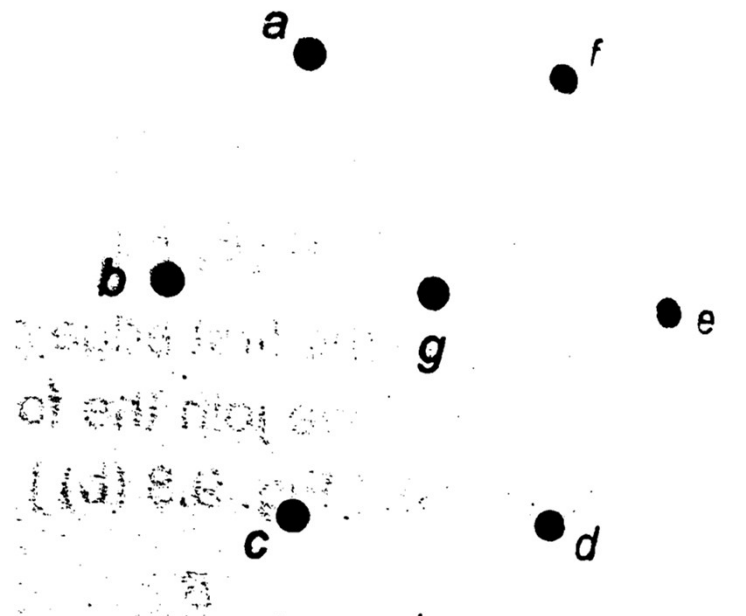
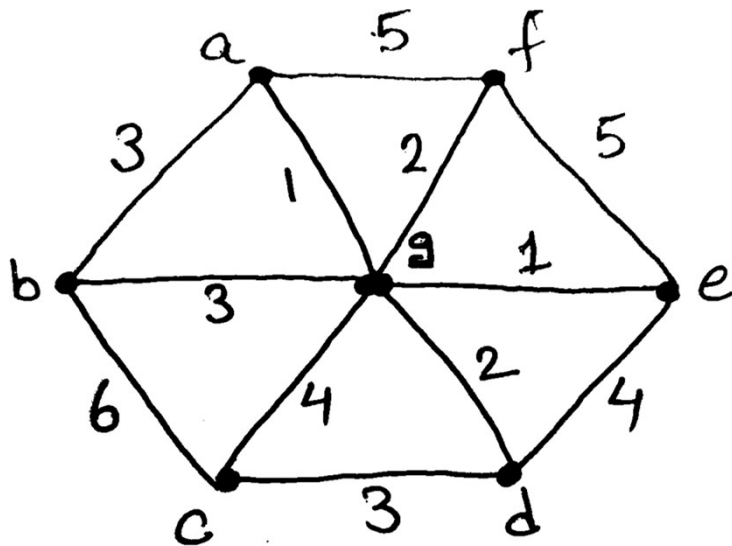
Example-2

Obtain minimal spanning tree and its weight for the graph shown below.



Example-2

Solution: we first name the vertices and jot down all of them as shown below



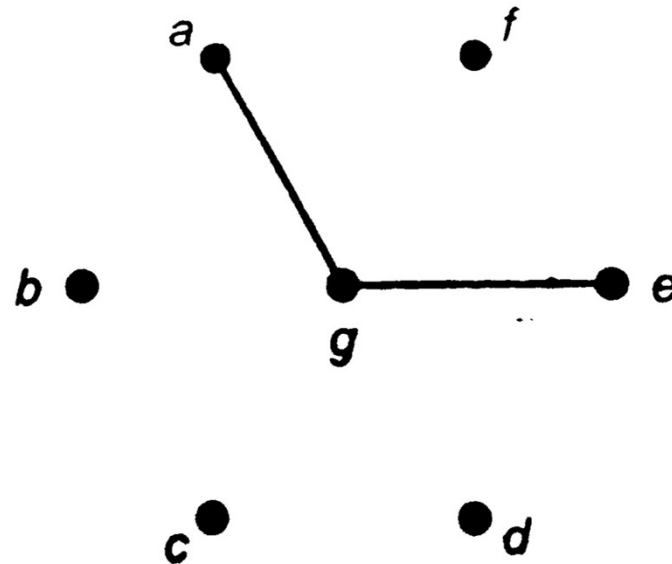
Example-2

We now list the edges in the increasing order of their costs.

Since the minimum cost is one we start with the edge $\{a, g\}$

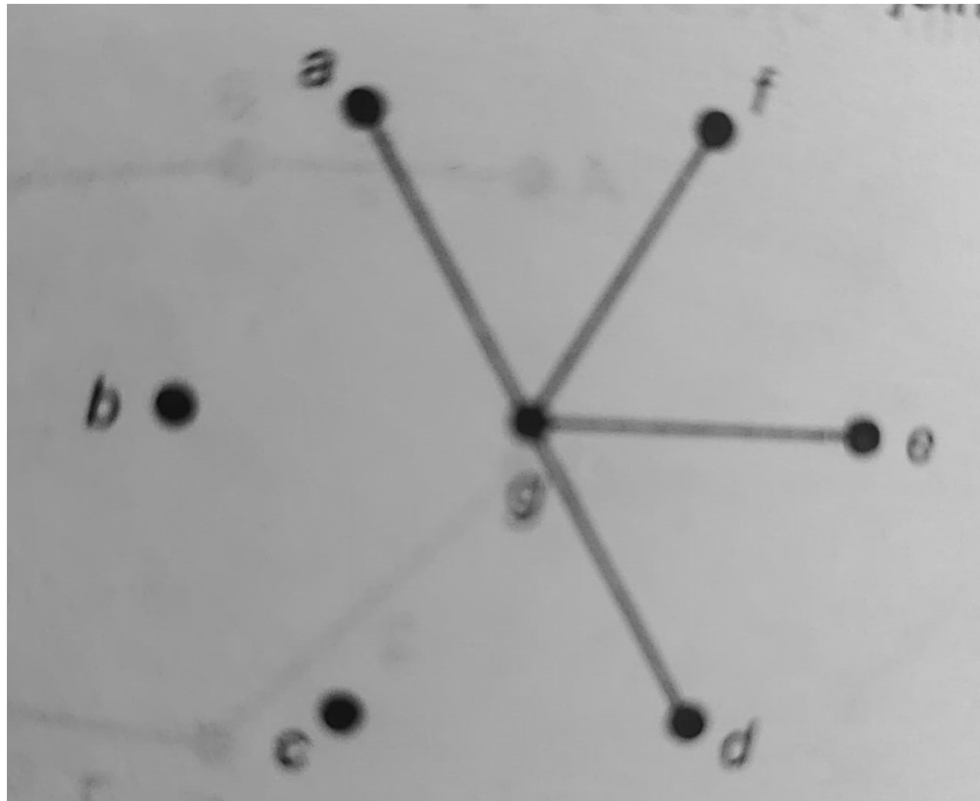
(1) $\{a, g\}$ (2) $\{g, e\}$ (3) $\{g, f\}$ (4) $\{g, d\}$ (5) $\{g, b\}$ (6) $\{b, a\}$
(7) $\{c, d\}$ (8) $\{c, g\}$ (9) $\{d, e\}$ (10) $\{e, f\}$ (11) $\{f, a\}$ (12) $\{b, c\}$

Step 1 : since there are two pairs of vertices $\{a, g\}$ and $\{g, e\}$ with minimum cost one, we join them.



Example-2

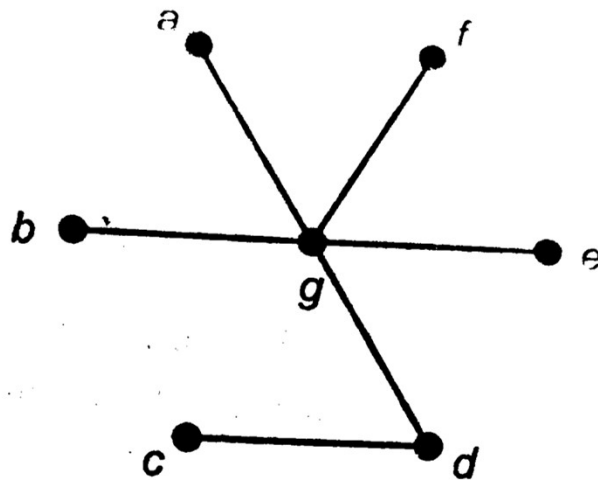
Step 2 : the two pairs of vertices having weight two are $\{g, f\}$ and $\{g, d\}$. Since they do not form cycles with the first two edges, we join them.



Step 3: there are three pairs of vertices having weight three.

They are $\{g, b\}$, $\{b, a\}$ and $\{c, d\}$.

We join the vertices g, b and c, d . But we do not join the vertices b, a because this edge forms a cycle. We reject this pair of vertices.



Since all the vertices are now joined this is the required minimum spanning tree. By adding the costs we get the minimal cost $= 1 + 1 + 2 + 2 + 3 + 3 = 12$.

Kruskal's Second Method

Kruskal's algorithm can be executed in the reverse order also.

The following steps are involved in this method.

First draw the given graph with the weights of the edges.

List all the edges in decreasing order of costs.

Step-1: Find the edge which has maximum cost and which forms a cycle. Delete this edge say e_1 and get the graph $G' = G \sim e_1$.

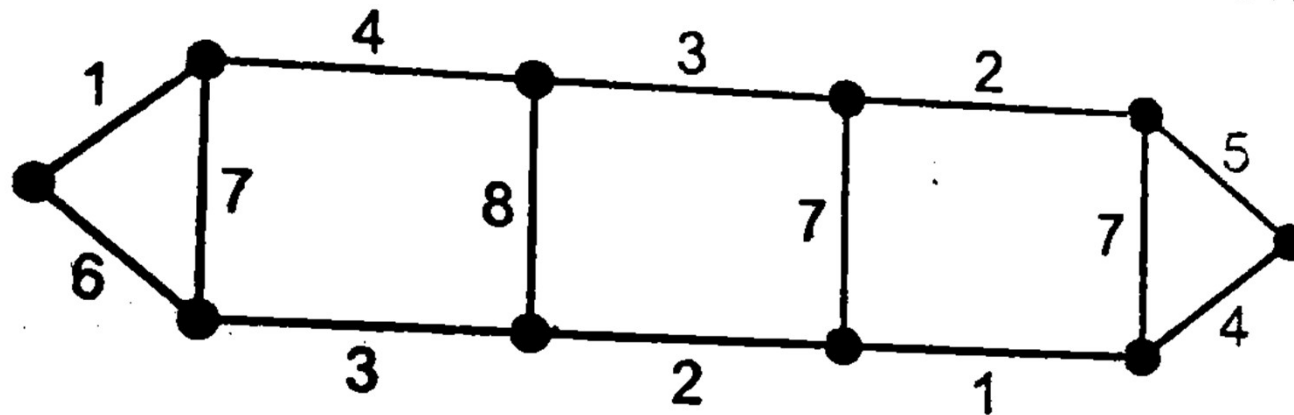
If G' is a tree the problem is solved. If G' is a cycle go to the step two.

Step-2: Find an edge which has maximum weight in G' and which forms a cycle in G' . Delete this edge say e_2 and get the graph $G'' = G' \sim e_2$.

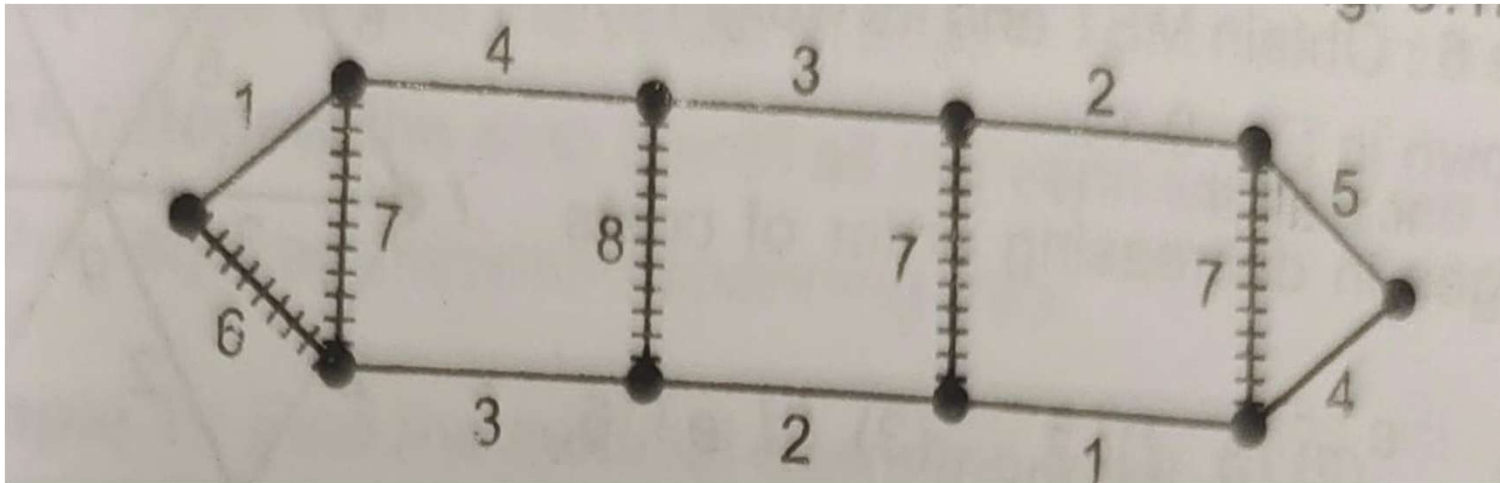
If G'' is a tree the problem is solved. If G'' is not a tree repeat the steps till you get M.S.T.

Example

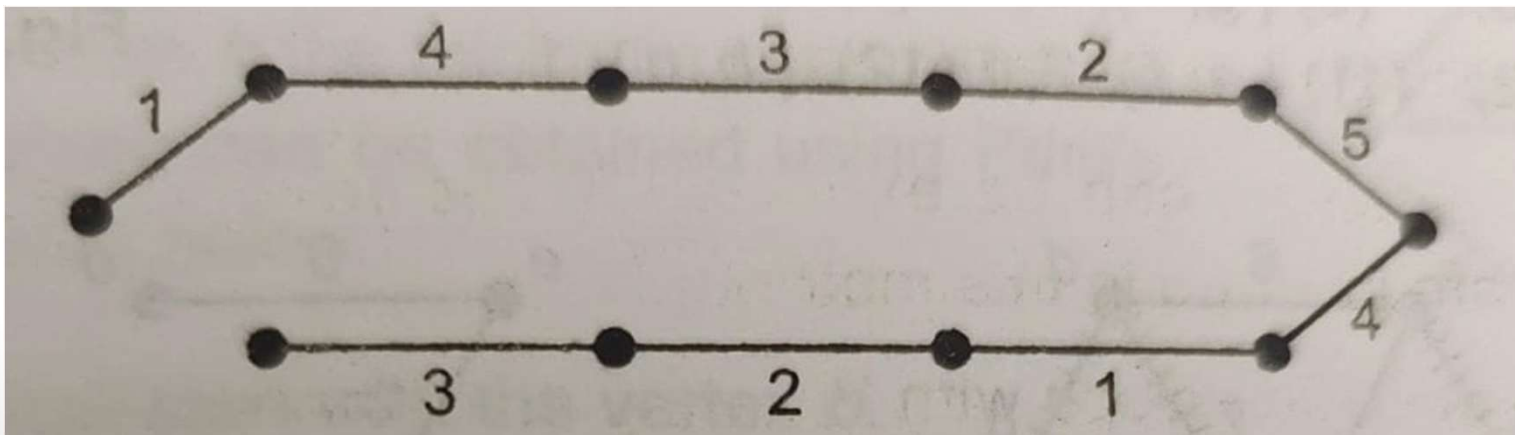
Obtain the minimum spanning tree for the following graph.



We cross the edges of weights 8,7,7,7,6 in that order to break the cycle at each stage.



We thus get the MST



Prim's or Disjkstra's Algorithm

Another way to compute minimum spanning tree for a weighted graph is Prim's or Disjkstra's Algorithm.

It was discovered by Prim and Dijkstra simultaneously but independently.

Step 1: Select any arbitrary vertex say v_0 as tree root.

Step 2: Find edge $e_1 = (v_0, v_1)$ in E such that its one end vertex v_0 is in T and its weight is minimum. Join the vertex v_0 to v_1

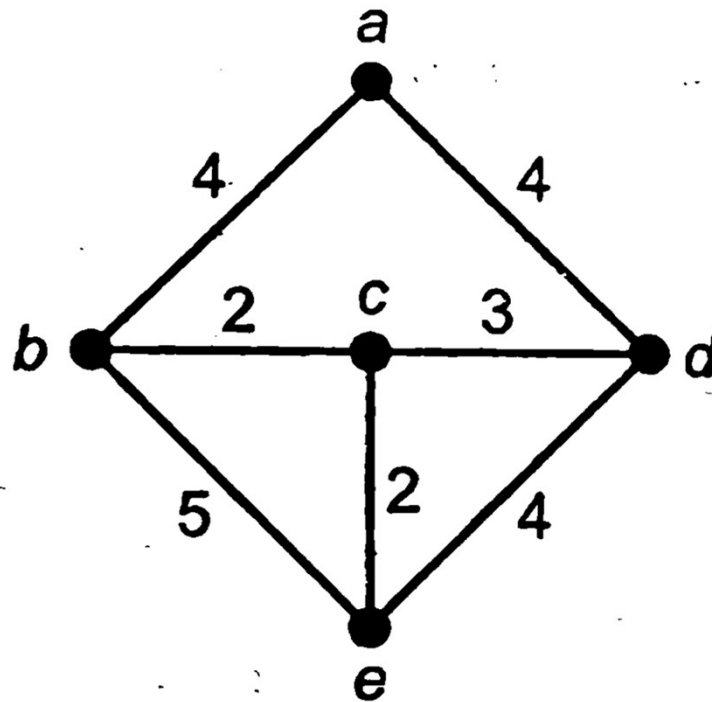
Step 3: Select the next edge $e_i = (v_i, v_j)$ such that its one end vertex v_i is in T and the other end vertex v_j is not in T and also the weight of the edge e_i is as small as possible. Join the edge e_i and the vertex v_j to T .

Step 4: Repeat the step 3 until all the vertices of G are in T .

T is the required minimum spanning tree.

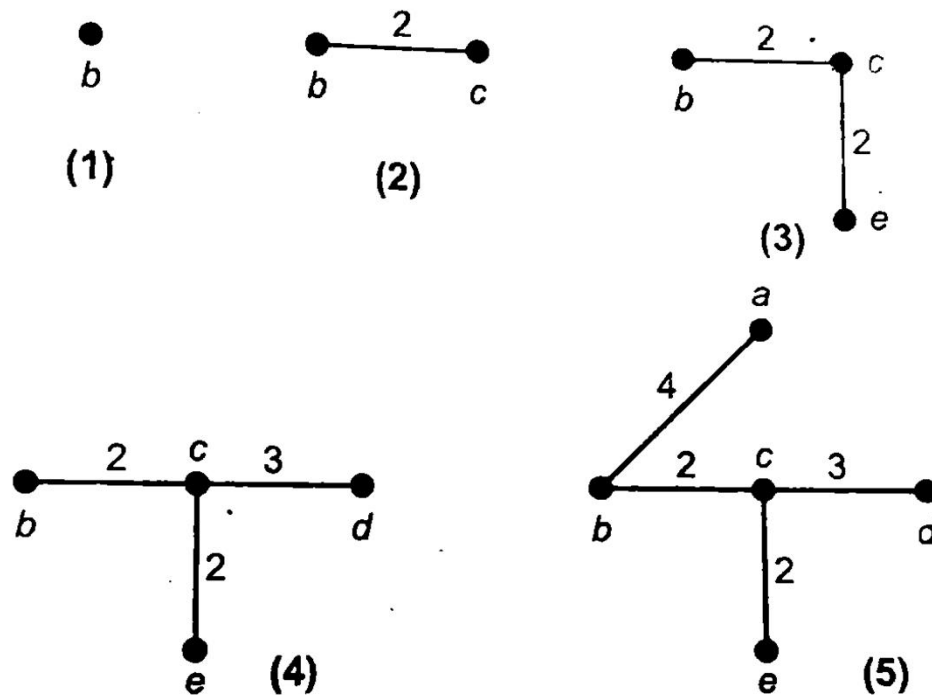
Example

- Find the minimal spanning tree for the adjoining graph using Prim's algorithm.



Example

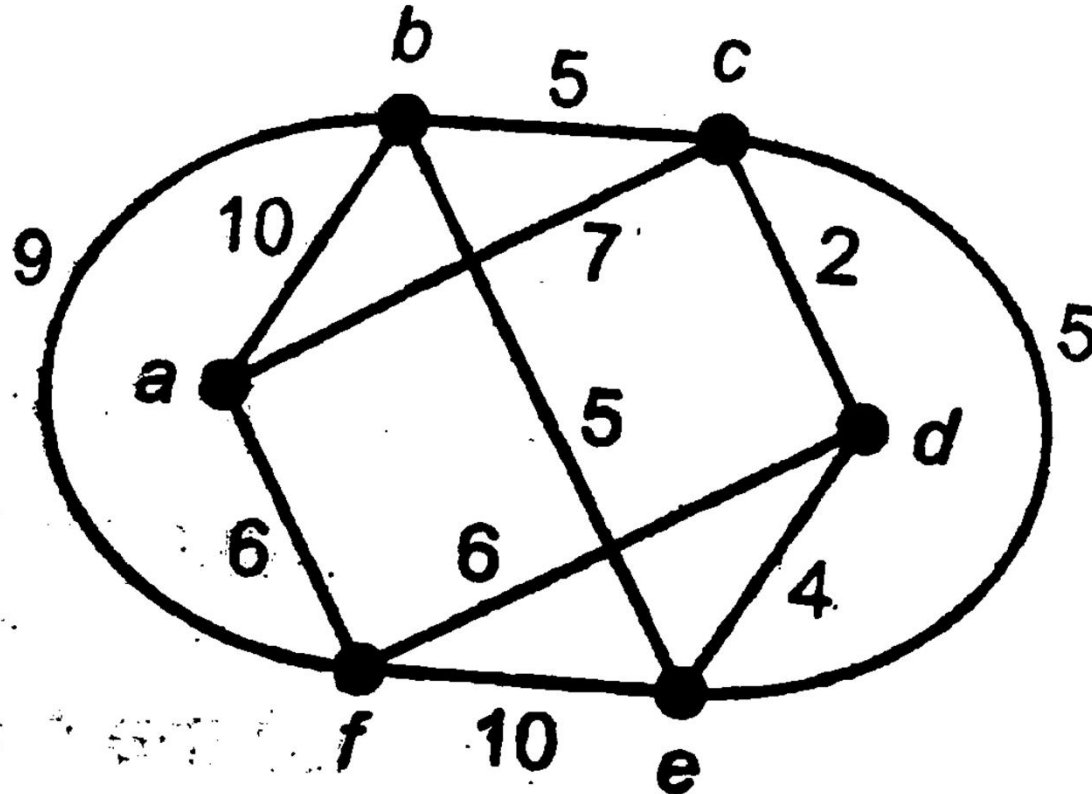
Solution: Starting from b the minimum spanning tree of the given graph can be obtained using Prim's algorithm is given below. We shall start with vertex b .



The graph obtained in the last step 5 is a minimum spanning tree. Its weight = $4+2+3+2=11$

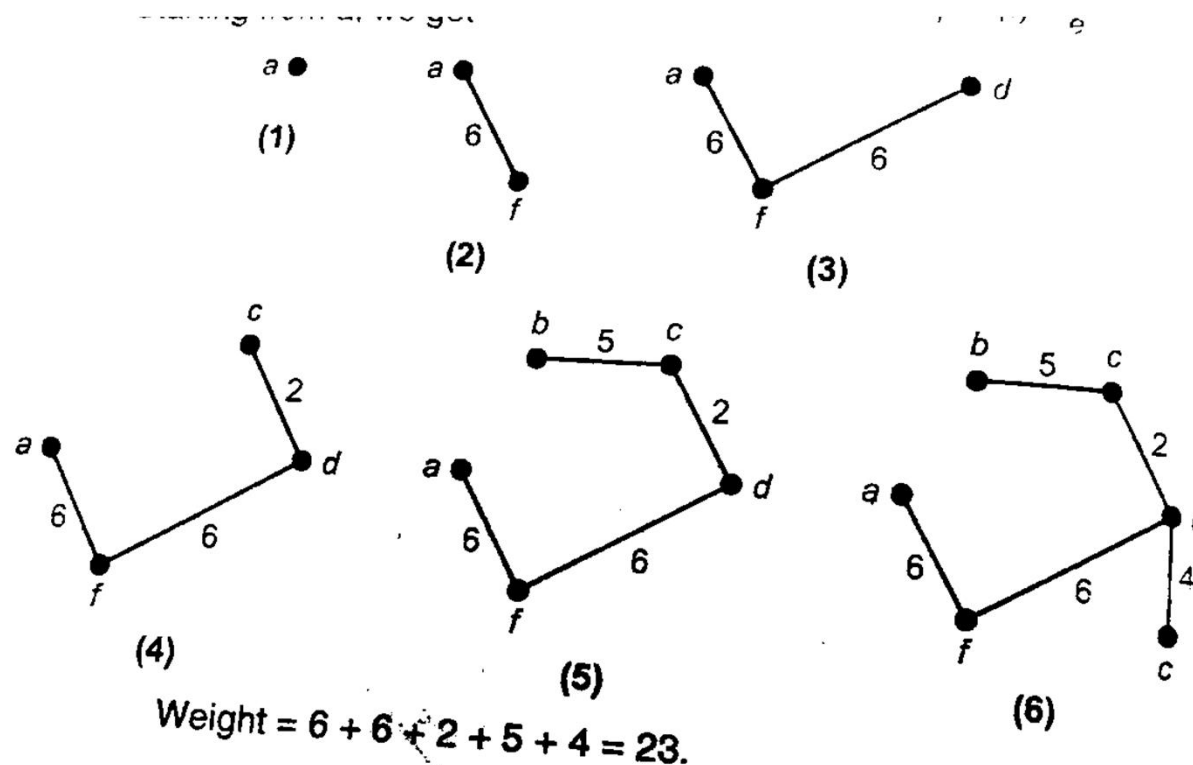
Example

- Find the minimal spanning tree for the adjoining graph using Prim's algorithm.



Example

Solution:



The graph obtained in the last step 6 is a minimum spanning tree. Its weight = $6+6+2+5+4 = 23$

Definition : Binary Tree

A tree having exactly one vertex of degree two, remaining vertices of degree one or three is called as binary tree.

Example :



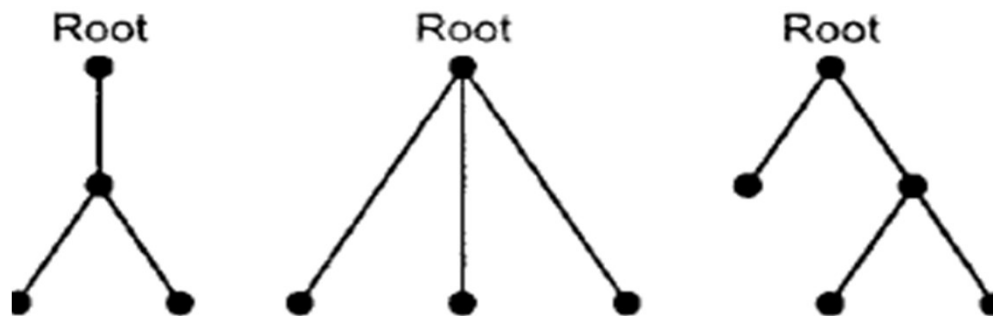
Definition :

A tree in which exactly one vertex is distinguishable from all other vertices is called a **rooted tree**.

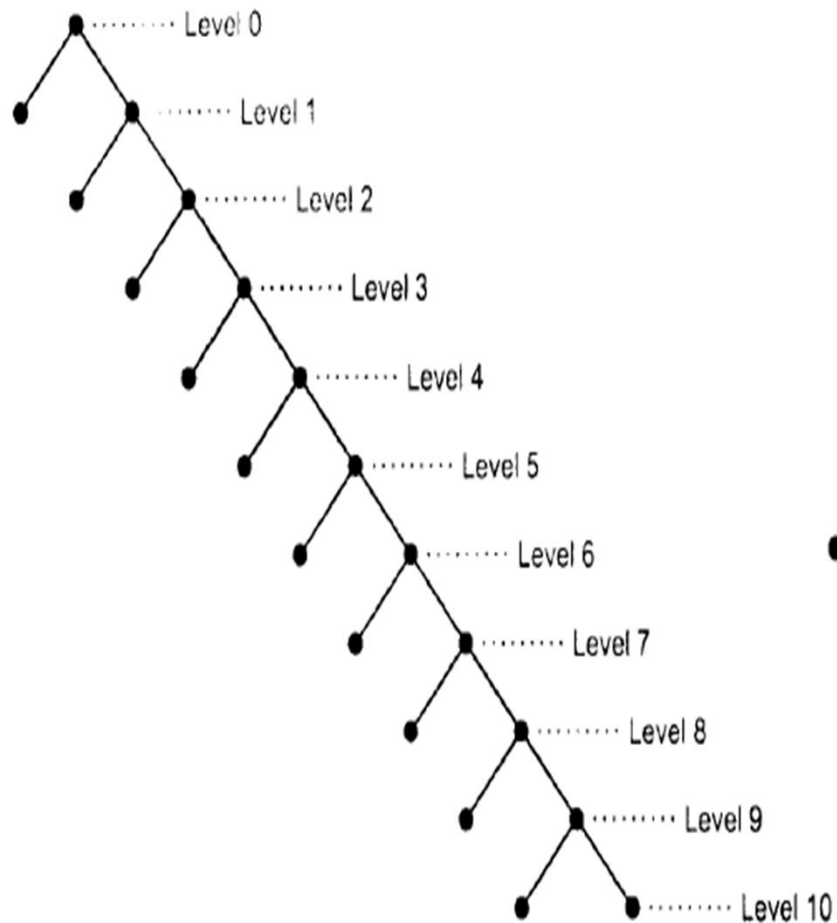
The particular distinguishable vertex is called the **root** of the tree.

Example

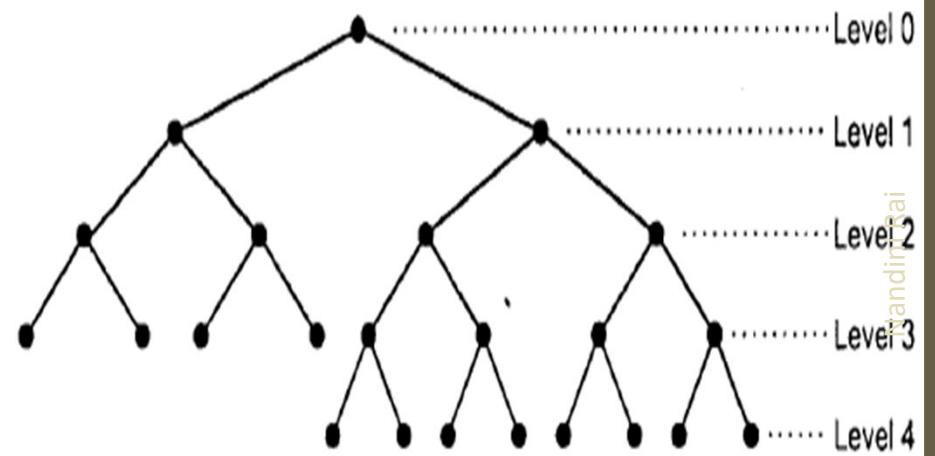
:



Graph:



- 1) Level of a vertex
- 2) Height of a Tree



Full binary tree: If in binary tree every internal node has exactly 2 branches it is called as full binary tree.

Pendant : In binary tree vertices of degree 1 are called as pendant vertices.

Non-pendant: In binary tree vertices of degree 3 & 2 are called as non- pendant vertices.

Property 1 : The number of vertices in binary tree is always odd.

Property 2 : The number of pendant vertices in binary tree with n vertices is $(n+1)/2$

Property 3 : The number of non-pendant vertices in binary tree with n vertices is $[(n+1)/2]-1$.

Property 4 : The maximum height of binary tree with n vertices is $(n-1)/2$ and minimum height is smallest integer greater than or equal to $\log_2(n + 1)-1$

Let $\text{ceil}(x)$ be the smallest integer greater than or equal to x , it is denoted by $\lceil x \rceil$ e.g. $\lceil 3.65 \rceil = 4$, $\lceil 0.35 \rceil = 1$

Example : Find maximum, minimum height of binary tree with 21 vertices

answer

$$\text{Maximum height} = \frac{21-1}{2} = 10$$

$$\text{Now, } \log_2 (21 + 1) - 1 = \log_2 22 - 1$$

$$= \frac{\log_{10} 22}{\log_{10} 2} - 1 = \frac{1.3424}{0.3010} - 1$$

$$= 4.4598 - 1 = 3.4598$$

$$\therefore \text{Minimum height} = \text{Smallest integer greater than } 3.4598 = 4$$

Example : Find maximum ,minimum height of binary tree with 11 vertices

Prefix Code

sequence

An ordered set is called as sequence

010 is a sequence using 0&1 of length 3

01110 is a sequence using 0&1 of length 5

Example :

$C = \{00, 001, 01, 010, 11, 111\}$

In the set C sequence 00 is prefix in (appears before) the sequence 001

$A = \{000, 001, 01, 10, 110, 111\}$

In the set A no sequence is prefix in (occurs before) any other sequence

$B = \{01, 10, 001, 110, 1111\}$

In the set B no sequence is prefix in any other sequence

Binary prefix code:

A set of sequences formed using digits 0 & 1 is called as a binary prefix code if no sequence is prefix of any other sequence in the set

Example :

$A = \{000, 001, 01, 10, 110, 111\}$

In the set A no sequence is prefix in any other sequence

$B = \{01, 10, 001, 110, 1111\}$

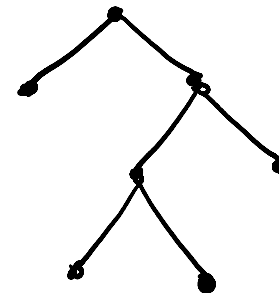
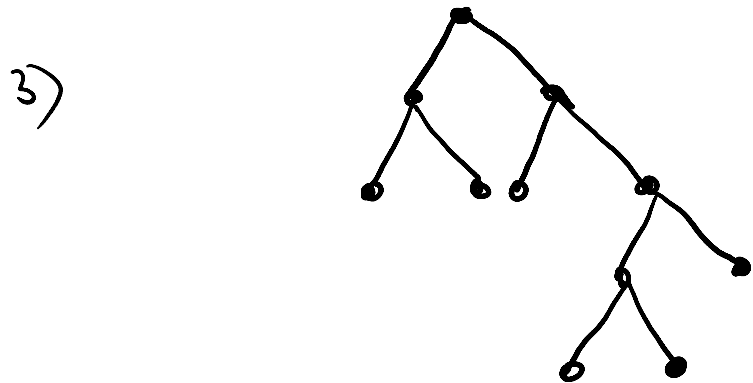
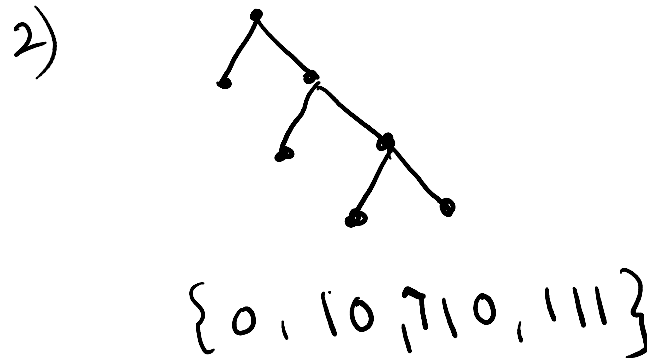
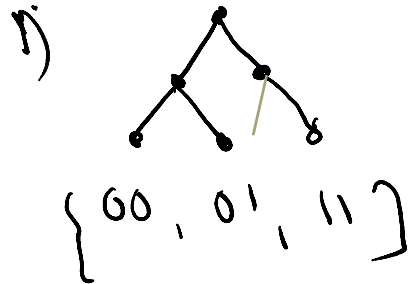
In the set B no sequence is prefix in any other sequence

Are A and B Binary prefix code?

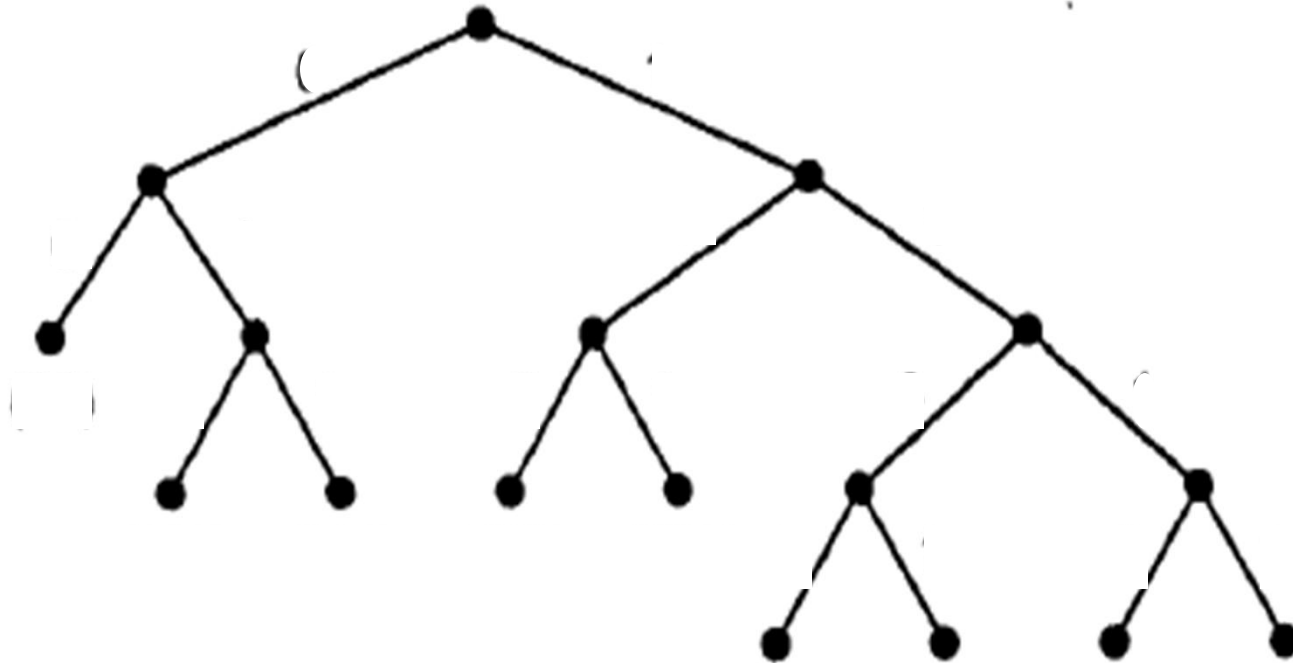
REMARK

Using digits 0 and 1, we can represent a binary tree by binary prefix code and conversely from a binary prefix code, we can construct a binary tree. Consider a full binary tree in which from each vertex including the root there are two branches. Let us agree to assign 0 to the branch going to the left and assign 1 to the branch going to the right. In this way we go on assigning numbers to all edges and reach the terminal nodes. Terminal nodes then can be represented by sequences of 0, 1 as shown in the following diagram.

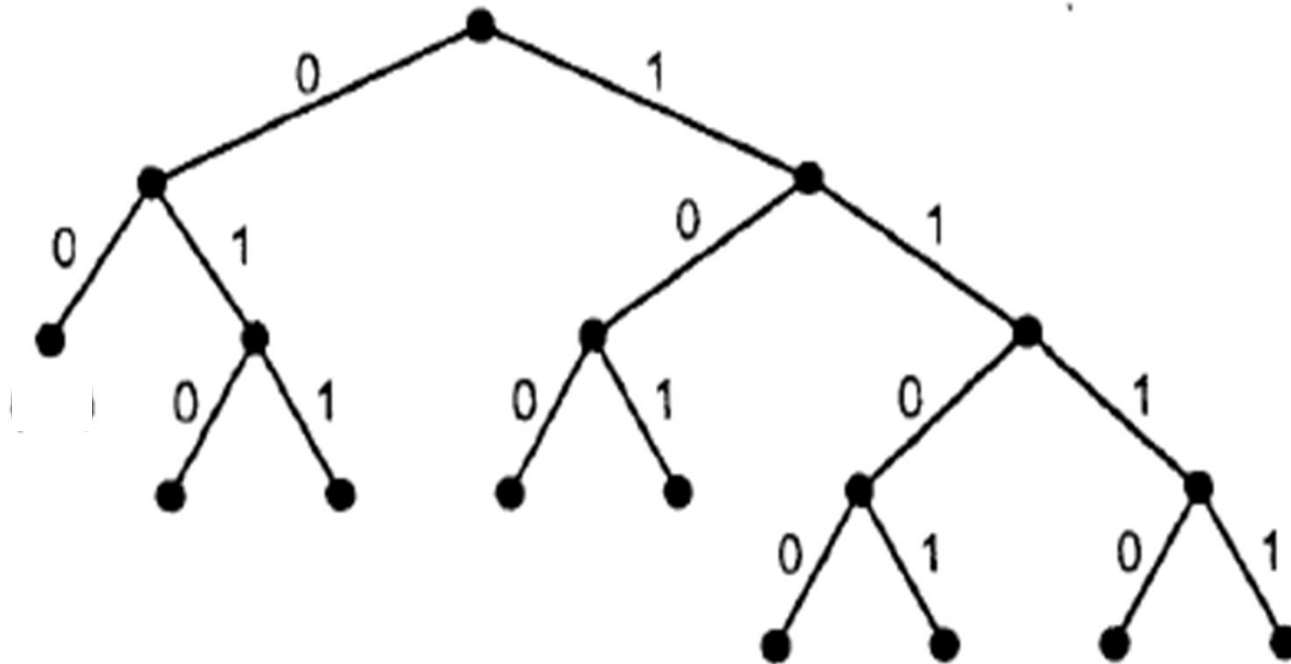
Example : Write binary prefix code for a given binary tree



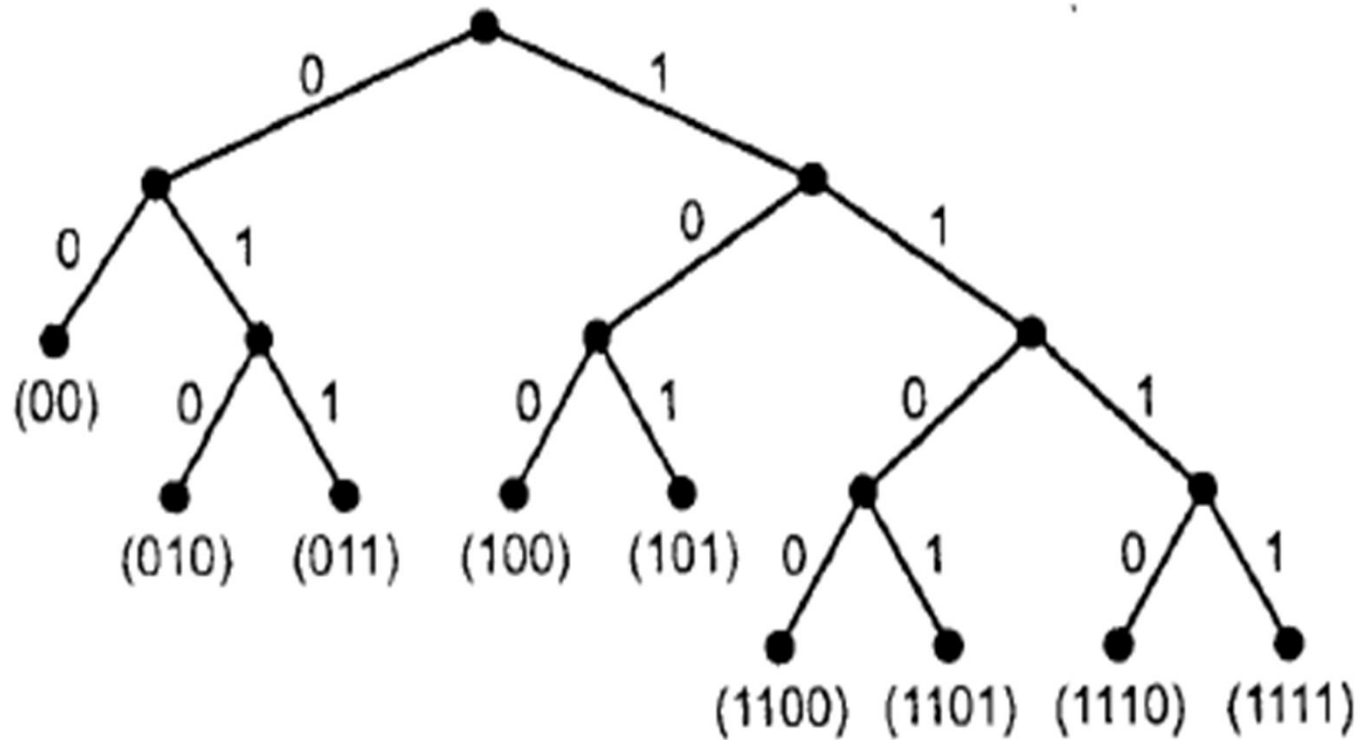
Example : Write binary prefix code for given binary tree



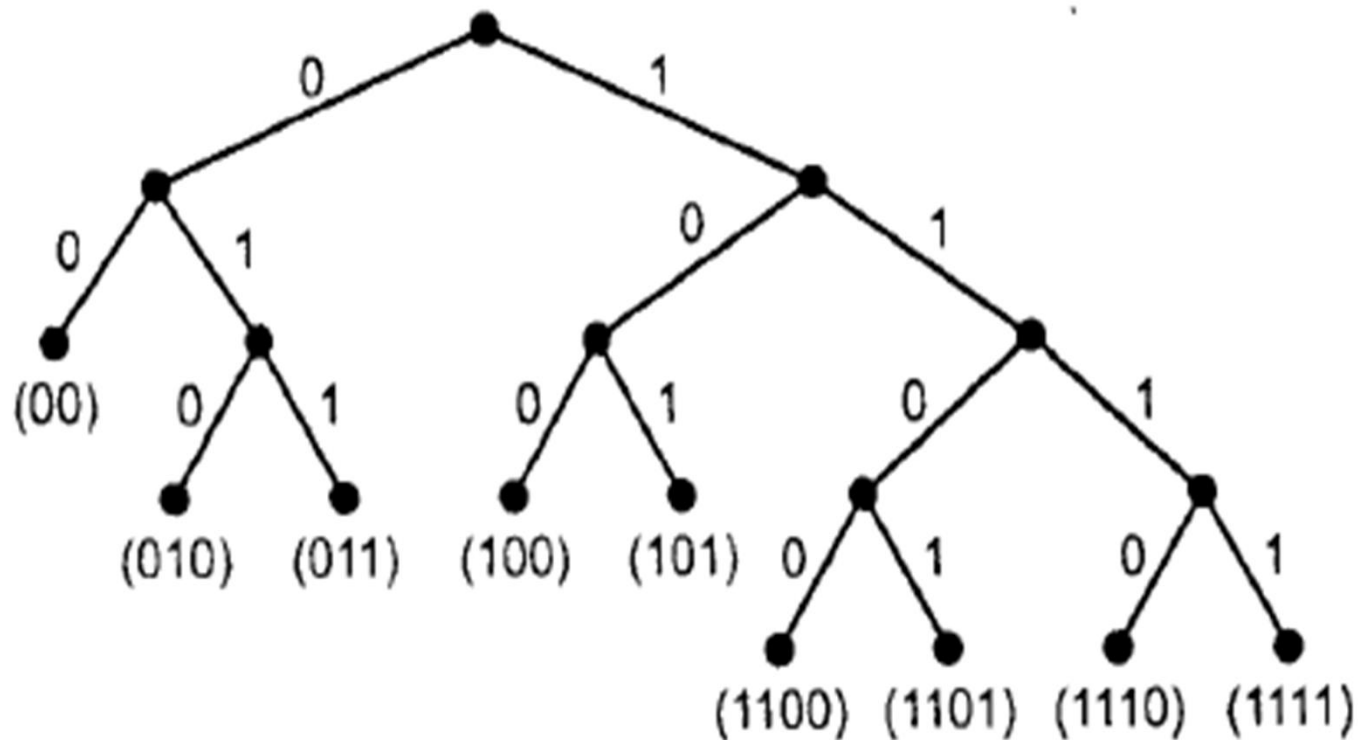
Example : Write binary prefix code for given binary tree



Example : Write binary prefix code for given binary tree



Example: Write binary prefix code for a given binary tree

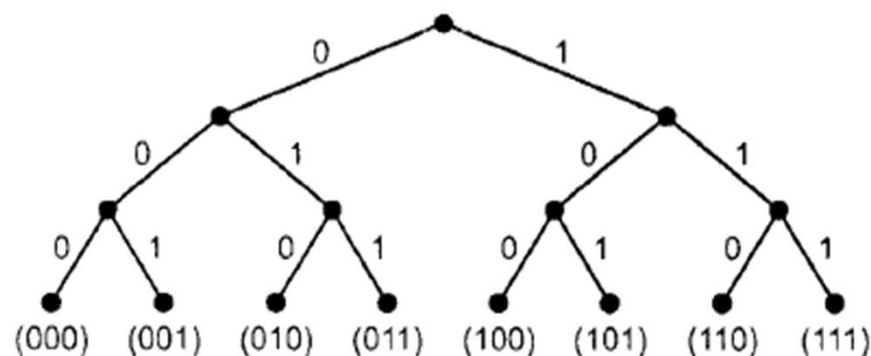


The prefix code of the above tree is

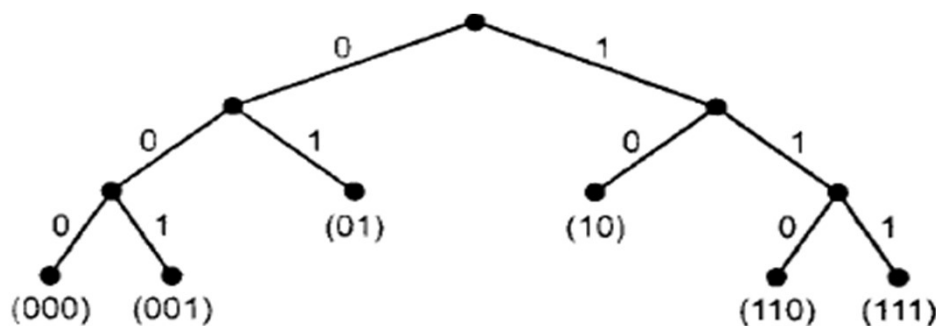
{ 00, 010, 011, 100, 101, 1100, 1101, 1110, 1111 }.

Example 1 : Draw a binary tree for binary prefix code {000,001,01,10,110,111}

Since the longest sequence is of length three we first construct a full binary tree of level three as shown below.

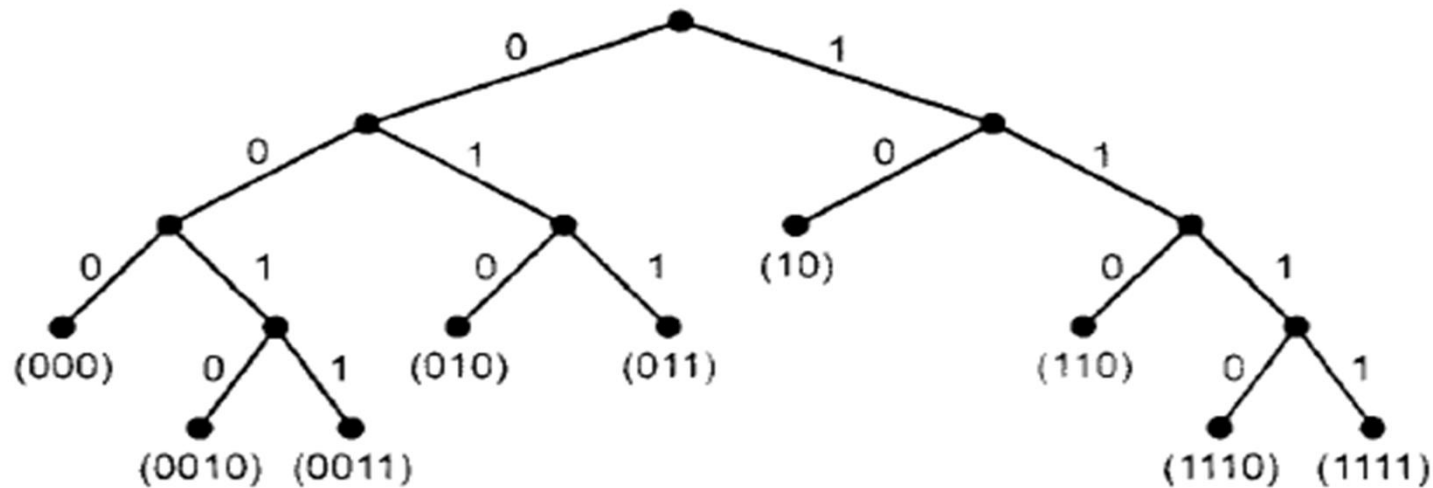
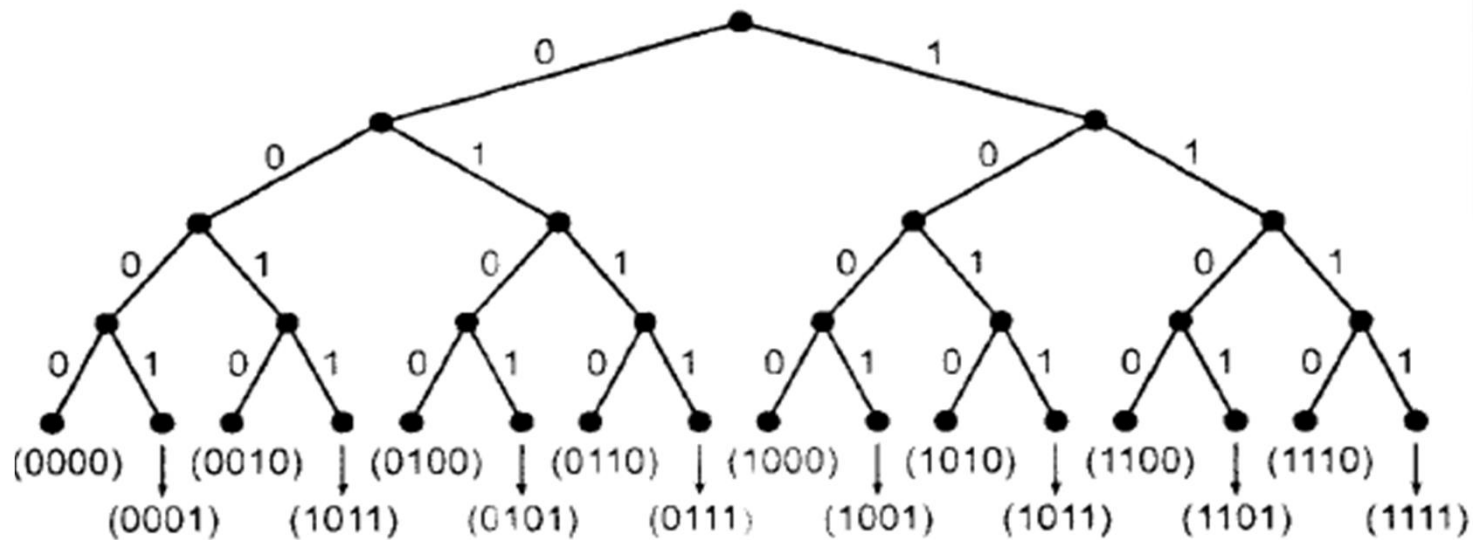


Now, we delete those vertices whose sequences are not included in the given prefix code. We also delete the incident edges. Thus, we get the following tree.



Example: binary prefix code for a binary tree is
{10,000,010,011,110,0010,0011,1110,1111} Then draw Tree

Example 2 : Draw a binary tree for binary prefix code {10,000,010,011,110,0010,0011,1110,1111}



Optimal Binary Tree

Weight of a Tree

If terminal nodes of binary tree are given some weights, Say $w_1, w_2, w_3, \dots, w_k$ whereas they are located at level $l_1, l_2, l_3, \dots, l_k$ respectively. Then Weight of the Tree is given by, $W = \sum_{i=1}^k w_i l_i$

Huffman Procedure to find optimal binary tree for a given set of weights

Example: Obtain a optimal binary tree for weights 4, 3, 2, 11, 7. Also find Weight of Optimal Binary Tree.

Example: Obtain an optimal binary tree for weights 4, 3, 2, 11, 7. Also find Weight of Optimal Binary Tree.

ANS: Step 1 – Arrange weights in ascending order 2, 3, 4, 7, 11

Step 2 – Form subtree for two smallest Weights i.e. 2, 3 and $2+3 = 5$

Step 3 – Now Consider 5, 4, 7, 11
Repeat Step 1 – 3.

Ascending - 4, 5, 7, 11

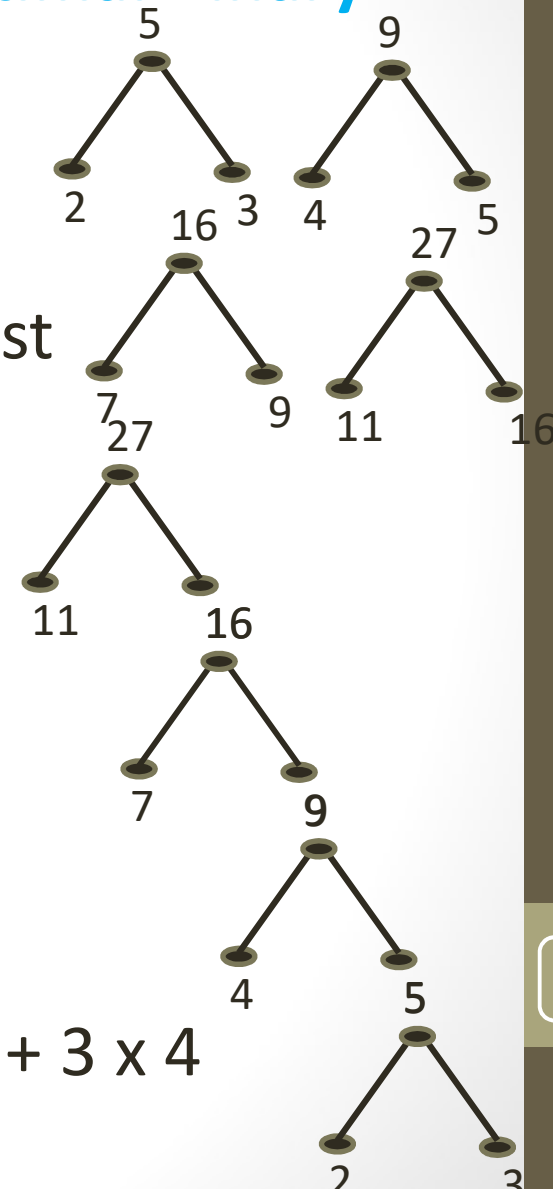
Subtree for 4, 5 and $4+5 = 9$

Consider now 9, 7, 11 and so on.

Finally connect all subtree properly.

Weight = $11 \times 1 + 7 \times 2 + 4 \times 3 + 2 \times 4 + 3 \times 4$

Of OBT = 57



Coding and Decoding Using Binary Tree

Example : Using the code given by the following table encode the words (i) earn, (ii) year, (iii) treaty.

Character	:	<i>o</i>	<i>a</i>	<i>e</i>	<i>n</i>	<i>r</i>	<i>t</i>	<i>y</i>
Code	:	1100	1101	01	1110	10	0	1111

Sol. : To encode a word we just go on writing the codes of the characters in the word serially.

(i) 'earn' is coded as code of e then code of a, then code of r and lastly code n.

01 1101 10 1110

(ii) 'year' is coded as y + e + a + r

i.e. 1111 01 1101 10

(iii) treaty is coded as t + r + e + a + t + y

i.e. 0 1 0 0 1 1 10 1 0 1111

Using the Haffman tree decode the message 01010011011110110

From the tree, we first prepare the table of the Huffman code of the characters.

Character	:	<i>u</i>	<i>c</i>	<i>f</i>	<i>o</i>	<i>e</i>	<i>d</i>
Code	:	00	010	011	10	110	111

since the first digit is 0, the first character is u, c or f.

Second digit is 1. Hence, c or f.

Third digit is 0, Hence c.

Fourth digit is 1. Hence, o, e or d.

Fifth digit is 0. Hence, o.

Sixth digit is 0. Hence, u, c or f.

Seventh digit is 1. Hence, c or f.

Eighth digit is 1. Hence, f.

Ninth digit is 0. Hence, u, c or f.

Tenth digit is 1. Hence, c or f.

Eleventh digit is 1. Hence, f.

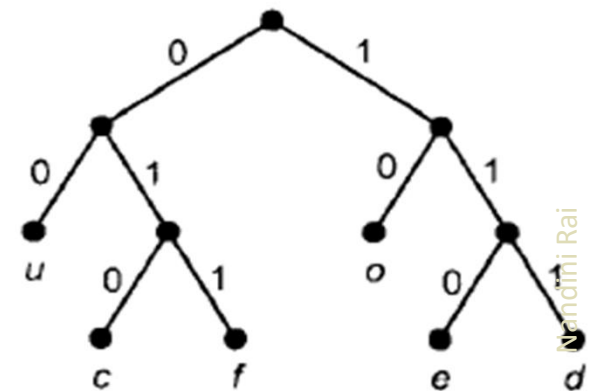
Twelfth digit is 1. Hence, o, e or d.

Thirteenth digit is 1. Hence, e or d.

Fourteenth digit is 0. Hence, e.

The last sequence is 110 again. Hence, e.

Hence, the message is 'coffee'.

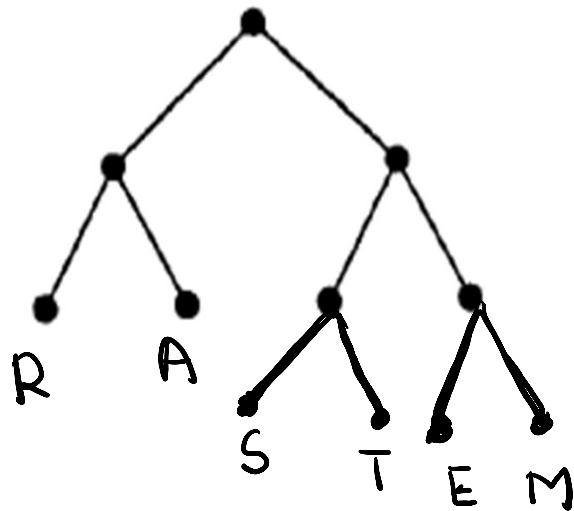


Example :

Using the given Huffman tree decode the message

(i) 1110110010111000

(ii) 10111001111



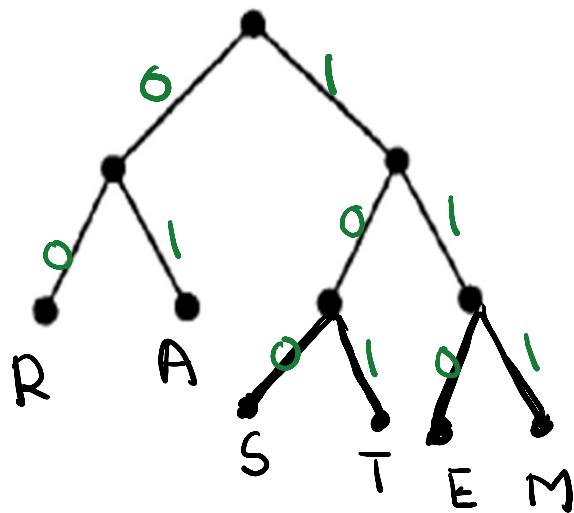
Example :

Using the given Huffman tree decode the message

(i) 1110110010111000

(ii) 10111001111

ANS



111 01100 101 11000
M A S T E R

Character code

R ↔ 00

A ↔ 01

S ↔ 100

T ↔ 101

E ↔ 110

M ↔ 111

1011100111
T E A M